

SALES ACQUISITION OPERATING SYSTEM

CAPABILITY DOCUMENT / CORPORATE EDITION

AcquisitionOS

Autonomous pipeline for modern revenue teams.

An end to end acquisition platform that unifies lead discovery, AI scoring, multi channel outreach, pipeline automation, and real time revenue intelligence inside one operating system.

STATUS / IN DEVELOPMENT

PREPARED FOR

Corporate Listing & Client Review

EDITION

2.0 / 2026

Confidential capability overview. Distribution restricted to authorised personnel.

CONTENTS

Table of Contents

Mission	4
Operating Principles	4
The Problem / Before	5
Where The Leakage Happened	5
What We Built	6
Module Inventory	6
System Architecture	9
Layer By Layer	9
Why A Layered System, Not Microservices Everywhere	10
Deployment Topology	10
Data Flow	11
The Full Lifecycle In Twelve Steps	11
Before and After	13
Operating Model	13
Time And Effort	13
What The Team Stops Doing	14
What The Team Starts Doing	14
Tech Stack	15
Frontend	15
Backend Application Server	15
Python Service Layer	16
AI And ML	16
Real Time And Messaging	16
Integrations	17
Infrastructure	17
Delivery Discipline	18
The Practices We Follow	18
Field Notes / Verified	20
What Is Verified Working	20

What Is In Progress	21
What Is Explicitly Out Of Scope Today	21
In Summary	22
At A Glance	22
Document Control	22

SECTION 01

Mission

AcquisitionOS exists to compress the distance between a prospect appearing on the open internet and a signed customer on the books. We believe that the act of acquiring customers should not require a dozen disconnected tools, a spreadsheet of copy paste workarounds, or a team of specialists stitching reports together by hand. The system replaces that fractured stack with a single, coherent operating layer that discovers leads, scores them, reaches out, books meetings, and reports revenue, all under the supervision of the people who own the outcome.

Our Commitment

To give every revenue team, from a two person founding crew to an enterprise sales floor, the same acquisition firepower that was previously reserved for companies with full revenue operations departments. Discovery, scoring, outreach, pipeline, and reporting are treated as one continuous workflow, not five tabs in five subscriptions.

Operating Principles

Single Source of Truth. Every lead, deal, message, and meeting lives in one data model. No silent drift between tools, no orphaned records, no reconciliation calls on Monday morning.

Autonomy With Oversight. The system is built to act on its own, to discover, score, draft, schedule, and follow up, but every autonomous action is logged, reversible, and observable to a human owner.

Cost Discipline. AI credits, scraping budget, and API spend are tracked per action and surfaced to the user. The platform is engineered to be transparent about what each acquisition actually costs.

Compliance First. GDPR consent, audit logs, data retention, and unsubscribe handling are wired into the core, not bolted on as a legal afterthought. The system is designed to be defensible from day one.

SECTION 02

The Problem / Before

Before AcquisitionOS, the typical revenue team operated a stack that no single person fully understood. A prospecting tool fed a CRM that fed a sequencing tool that fed a calendar that fed a reporting layer that fed a billing system, each one a separate login, a separate bill, and a separate source of truth. The seams between those tools were where revenue leaked. Leads fell through gaps. Scores were guessed. Follow ups were forgotten. Reports were assembled by hand. Meetings were booked twice or not at all. The team spent more energy operating the stack than operating the pipeline.

Where The Leakage Happened

Stage	What Teams Did By Hand	What Slipped Through
Discovery	Manual list building from LinkedIn, directories, search results	Stale data, duplicate records, missed niches
Scoring	Gut feel or static spreadsheets ranking who to call first	Hot leads left cold, cold leads over pursued
Outreach	Copy paste templates across Gmail, WhatsApp, Telegram	Inconsistent messaging, no A/B signal, reply loss
Pipeline	Kanban boards updated once a week from memory	Stage drift, forgotten follow ups, silent churn
Meetings	Email ping pong to find a slot, manual calendar holds	Double bookings, no shows, lost context
Reporting	Friday night spreadsheet from three exports	Stale numbers, no anomaly detection, reactive not predictive
Billing	Manual invoicing, manual credit tracking, manual tax	Revenue leakage, compliance risk, dispute exposure

The Honest Summary

The problem was never that individual tools were bad. The problem was the integration tax. Every handoff between tools cost time, lost context, and introduced error. A team of five could easily spend 40 percent of its week simply moving data between systems instead of moving prospects through a pipeline.

SECTION 03

What We Built

AcquisitionOS is one platform, one database, one login. It collapses the scattered revenue stack into a single operating system organised around the natural lifecycle of a customer. The architecture is intentionally monolithic at the data layer, every record knows about every other record, and intentionally modular at the service layer, so each capability can be hardened and scaled on its own. What follows is a plain language inventory of what the system actually does today.

Module Inventory

Lead Discovery & Sourcing. Automated discovery jobs scan company websites, score online footprint, run competitive gap analysis, and surface prospect lists by niche, geography, and intent signals. Distributed scraping with proxy rotation and anti bot handling keeps discovery resilient at scale.

AI Lead Scoring & Enrichment. Every lead is scored on a multi signal model that blends firmographics, website quality, reply intelligence, and engagement history. Enrichment fills missing fields from public sources. The scoring engine explains its own rankings, so a human reviewer can see why a lead is hot.

Pipeline & Deal Management. A configurable pipeline with custom stages, deal risk assessment, revenue waterfall, and forecast modelling. Deals link to their originating leads, every activity is auditable, and stage movements fire downstream automations.

Outreach & Sequences. Multi step email sequences, autonomous outreach campaigns, message broadcasts, and a template approval workflow. Open and click tracking, bounce intelligence, and unsubscribe compliance are built in. Templates support variables, conditional blocks, and per channel formatting.

Gmail Integration. Native Gmail sync via OAuth with inbox threading, reply processing, draft generation, send and receive, tracking pixels, and PubSub push notifications. Multiple Gmail accounts can be linked and switched, each isolated per user.

Calendar & Meeting Intelligence. Google Calendar integration with AI booking, availability detection, meeting prep, agenda generation, action item extraction, sentiment analysis, follow up email drafting, and objection handling. A meeting assistant turns the meeting lifecycle into a workflow, not a chore.

AI Copilot & RAG. An in dashboard AI assistant with streaming chat, retrieval augmented generation over ingested documents and URLs, vector search across the knowledge base, prompt management, and a credit enforced cost model. The copilot can analyse a lead, generate outreach, and answer

questions about pipeline health.

Workflow Engine. A visual workflow builder with triggers, conditions, actions, executions, retries, dead letter queues, and per workflow metrics. Workflows connect every other module, so a reply in Gmail can move a deal in the pipeline and fire a Telegram alert without any glue code.

Competitor Intelligence. Competitor discovery, SEO analysis, pricing snapshots, review monitoring, social signal tracking, and opportunity gap detection. Competitor data is stored as time series snapshots, so the team can see how a rival evolved over months, not just today.

Hot Lead Detection. A real time hot lead feed that watches reply signals, engagement spikes, and buying intent markers. The scanner runs on a schedule and surfaces leads that have crossed a heat threshold, so the team works the warmest opportunities first.

Autonomous SDR Pipeline. A self driving outreach pipeline that can generate, dispatch, classify replies, move pipeline stages, and send follow ups without manual prompting. Every autonomous action is logged and reversible, and the user sets the autonomy ceiling.

Billing, Credits & Invoices. Subscription tiers, credit ledger, addon packs, coupons, tax calculation, invoice generation and delivery, GST handling, Stripe and Razorpay payment rails, webhook reconciliation, dunning, and a customer portal. The billing layer is its own auditable sub system.

Team, Org & White Label. Organisations, team invites, role based access control, white label branding, audit logs, and an admin console. The system is built for a single founder and for a multi tenant agency under the same roof.

Integrations Hub. Gmail, Google Calendar, Telegram, WhatsApp (Meta Cloud API and Twilio), Stripe, Razorpay, and Resend, each with connect, disconnect, status, and health monitoring. The integration marketplace is the front door to the outside world.

Security & Compliance. GDPR export and delete, consent tracking, data retention policies, audit logs, API keys with rotation and revocation, session management, MFA, rate limiting, device fingerprinting, suspicious login detection, and a security audit trail.

Observability. Metrics, tracing, structured logging, Sentry integration, API monitoring, error tracking, health checks, and a real time observability dashboard. The system watches itself as carefully as it watches the pipeline.

Real Time Layer. WebSocket and Server Sent Events channels push live activity, notifications, and pipeline updates to the dashboard without polling. A dedicated real time mini service keeps every connected client in sync.

Notifications & Messaging. A multi channel notification engine that can deliver via email, push, Telegram, and WhatsApp from a single event. Notification preferences, broadcast history, and a notification center live in the dashboard.

Reporting & Analytics. An advanced report builder with templates, scheduling, export, and shareable links. Analytics cover lead sources, funnel velocity, deal performance, team workload, revenue forecast, benchmarks, anomalies, and executive summaries.

One Line Description

AcquisitionOS is the operating system that a revenue team would build for itself if it had the time, the engineers, and the patience to replace its entire tool stack with a single, coherent, accountable platform.

SECTION 04

System Architecture

The platform is a layered system. At the bottom sits a single relational database that holds every domain object, from users and leads to deals, meetings, workflows, and invoices. Above the database, a TypeScript application server exposes a large surface of HTTP API routes, one route per responsibility, organised by domain. Alongside the main server, a set of independent mini services handle concerns that benefit from process isolation, real time push, email delivery, and a watchdog. At the top, a React based single page application renders the dashboard and the public marketing surface. A gateway sits in front of everything, routing external traffic to the right internal port and shielding the application servers from direct exposure.

Layer By Layer

Layer	Responsibility	Technology
Presentation	Dashboard, auth pages, public marketing, legal pages, admin console	Next.js 16 App Router, React 19, Tailwind CSS 4, shadcn/ui, Framer Motion, TanStack Query, Zustand, Recharts
API Surface	One HTTP route per domain action, rate limited, audited, session enforced	Next.js Route Handlers, TypeScript, Zod validation, JWT sessions, RBAC middleware
Domain Logic	Services that encode the business rules for each module	TypeScript service classes, Prisma ORM, bcrypt, Nodemailer, Stripe and Razorpay SDKs, Resend SDK, z-ai-web-dev-sdk
Persistence	Single source of truth for every record	Prisma ORM over SQLite (production schema is PostgreSQL ready with migration annotations)
Real Time	Live updates to every connected dashboard client	Socket.io mini service, Server Sent Events, an in process event bus
Background Work	Cron driven jobs and long running tasks	Next.js cron routes, a server watchdog, a workflow executor with dead letter queues
AI / ML	Lead scoring, outreach generation, reply classification, RAG, vector search, copilot chat	z-ai-web-dev-sdk for LLM, VLM, TTS, ASR, and image generation, an in process vector index, RAG ingestion, a prompt manager, credit enforcement, an AI cost tracker

Layer	Responsibility	Technology
Python Service Layer	Heavy scraping, ML model serving, and data science pipelines that benefit from the Python ecosystem	FastAPI microservices, Scrapy and BeautifulSoup for scraping, Pandas and NumPy for data shaping, scikit-learn for scoring models, Celery with Redis for async tasks, OpenAI and Anthropic SDKs, LangChain for chains and agents
Integrations	Third party connections	Gmail API, Google Calendar API, Telegram Bot API, WhatsApp Meta Cloud API, WhatsApp via Twilio, Stripe, Razorpay, Resend, Google Custom Search, Sentry
Gateway	External entry point, TLS termination, port routing	Caddy reverse proxy, XTransformPort routing for mini services
Observability	System watches itself	Structured logging, metrics collector, request tracing, Sentry, health checks, API monitor, error tracker

Why A Layered System, Not Microservices Everywhere

A fully microservice architecture buys scale at the cost of operational complexity. For an acquisition platform, the most expensive failure is a lost or inconsistent record, a lead that exists in one service but not another, a deal that moved in the pipeline but the notification service never saw it. The platform keeps a single database and a single application server for the domain core, so every write is atomic and every read is consistent. Only the concerns that genuinely need isolation, real time push, email delivery, and the Python AI and scraping layer, are split into their own processes. The result is a system that scales to the needs of a real revenue team without requiring a site reliability engineering department to keep it upright.

Deployment Topology

The main application server runs as a standalone Next.js production build on port 3000, with the real time service on its own port, the email service on its own port, and the Python service layer on its own assigned ports. A single Caddy gateway listens on the public interface, terminates TLS, and forwards to the correct internal port using a query parameter. This topology means a new Python microservice can be added without touching the gateway configuration, it is discovered by its port and routed dynamically. The same property lets the system run on a single host for a small team or spread across hosts for a larger one, with no code changes, only configuration.

SECTION 05

Data Flow

Data moves through AcquisitionOS along a predictable path. A prospect enters the system through discovery, is enriched and scored, is enrolled in an outreach sequence, replies are processed and classified, a meeting is booked, a deal is created, the deal moves through pipeline stages, and finally revenue is recognised and invoiced. At every step, the data is written to the central database, events are fired to the real time layer, and downstream subscribers react. The flow is observable end to end, no step is silent.

The Full Lifecycle In Twelve Steps

01 Discovery. A discovery job, either scheduled or manually triggered, queries public sources via the scraping layer (Python Scrapy workers behind the gateway). Raw prospects are written to a staging table with their source URL, niche, and discovery timestamp.

02 Enrichment. The enrichment service fills missing fields, company name, website, location, employee range, using the company researcher module. Duplicates are deduplicated against the existing lead store by domain and email hash.

03 AI Scoring. The scoring engine pulls firmographics, website quality, reply intelligence, and engagement history, sends a structured prompt to the LLM, and writes a numeric score plus a human readable explanation back to the lead record.

04 Pipeline Enrollment. The lead is assigned to a pipeline stage, either by an automation rule or a human owner. The stage assignment fires a workflow trigger that can notify, schedule, or hand off to a sequence.

05 Sequence Enrollment. The lead is enrolled in an outreach sequence. The sequence executor schedules each step, drafts an email with the AI outreach generator, and waits for the scheduled send time.

06 Send. At the scheduled time, the email service delivers the message via the configured provider, Resend, Gmail SMTP, or Ethereum in test. Open and click tracking pixels are attached. The send is recorded against the lead.

07 Reply Intake. A reply arrives in Gmail. The PubSub push notification wakes the reply processor, which fetches the thread, classifies the reply with the reply intelligence module (interested, out of office, unsubscribe, objection), and updates the lead status.

08 Hot Lead Detection. If the reply signals buying intent, the hot lead scanner flags the lead and pushes a real time notification to the dashboard, to email, to Telegram, and to WhatsApp based on the

user notification preferences.

09 Meeting Booking. The user, or the autonomous pipeline on the user behalf, opens the schedule dialog. The calendar intelligence module reads availability from Google Calendar, proposes slots, and sends an invite. A meeting record is created and linked to the lead.

10 Meeting Execution. Before the meeting, the assistant generates a prep brief and an agenda. After the meeting, it extracts action items, detects sentiment, drafts a follow up email, and records objections. Every artefact is stored against the meeting.

11 Deal Creation. When the prospect agrees to terms, a deal is created from the lead. The deal enters the pipeline at the first stage. The revenue forecast, waterfall, and risk assessment all update in real time.

12 Revenue Close. On deal close, the billing service generates an invoice, applies tax and coupon logic, and charges via Stripe or Razorpay. The webhook reconciles the payment, the credit ledger updates, and the deal stage moves to won. The cycle is complete.

The Property That Makes It Work

Every step writes to the same database and fires an event to the same bus. There is no batch sync, no overnight reconciliation, no shadow copy of the data in a separate tool. When a lead moves, the pipeline moves. When the pipeline moves, the forecast moves. When the forecast moves, the dashboard moves. All in the same second.

SECTION 06

Before and After

The clearest way to describe what AcquisitionOS changes is to put the old way and the new way side by side. The comparison below is not aspirational, it describes the operating model the system is built to deliver.

Operating Model

Dimension	Before	After
Tools in use	6 to 10 separate subscriptions	1 platform, 1 login, 1 bill
Lead discovery	Manual list building, weekly	Continuous automated discovery jobs
Lead scoring	Gut feel, static spreadsheet	Multi signal AI model with explanations
Outreach	Copy paste across channels	Sequences with AI drafted, tracked steps
Reply handling	Inbox monitored by a human	Auto classified, routed, and flagged
Meeting booking	Email ping pong	AI reads calendar, proposes slots, sends invite
Pipeline visibility	Updated weekly from memory	Live, real time, auditable
Reporting	Friday spreadsheet from three exports	Always on dashboard, scheduled reports
Billing	Manual invoicing and tax	Automated invoicing, tax, and reconciliation
Compliance	Reacted to when asked	Wired into every record from day one
Onboarding a new rep	Days of tool training	A login and a role

Time And Effort

6 to 1 TOOLS PER REP Subscriptions collapsed into one	70% LESS COPY PASTE Across the outreach and reporting cycle	0 RECONCILIATION CALLS Single source of truth removes them
--	--	---

What The Team Stops Doing

- ✗ Moving CSV exports between five tools every Friday afternoon.
- ✗ Manually looking up a prospect on LinkedIn, then their website, then their competitors.
- ✗ Guessing which lead to call first because there is no score and no context.
- ✗ Asking the prospect for a time slot and then negotiating over four emails.
- ✗ Rebuilding the pipeline board in the weekly meeting because nobody updated it during the week.
- ✗ Reconciling the billing spreadsheet against the CRM at the end of the month.
- ✗ Explaining to a new hire which tool does what and how they connect.

What The Team Starts Doing

- ✓ Reviewing an AI generated hot lead feed every morning instead of building lists.
- ✓ Reading the scoring explanation before a call, so the conversation starts informed.
- ✓ Approving AI drafted outreach instead of writing it from scratch.
- ✓ Watching the pipeline move itself as replies are classified and stages advance.
- ✓ Opening a dashboard that already knows the answer instead of building a report.
- ✓ Onboarding a new rep with a login and a role, not a week of tool training.

SECTION 07

Tech Stack

The stack is chosen for a specific reason: it must be operable by a small team, auditable by an enterprise buyer, and portable across cloud providers. No component is exotic. Every layer is mainstream, well documented, and has a long support horizon. The platform is built to outlive any single hosting provider or vendor relationship.

Frontend

- ▶ **Next.js 16** with the App Router, the current generation of the React framework, chosen for server rendering, route handlers, and standalone output.
- ▶ **React 19** for the component model, with concurrent features and server components where they reduce client bundle size.
- ▶ **TypeScript 5** end to end, with strict typing. No untyped boundary between the frontend and the API.
- ▶ **Tailwind CSS 4** for the styling system, paired with the **shadcn/ui** component library in the New York style.
- ▶ **Framer Motion** for transitions and micro interactions, kept subtle and purposeful.
- ▶ **TanStack Query** for server state, **Zustand** for client state, and **React Hook Form** with **Zod** for forms and validation.
- ▶ **Recharts** for data visualisation, **Lucide** for icons, **cmdk** for the command palette, **sonner** for toasts.
- ▶ **MDX Editor** for rich text, **react-syntax-highlighter** for code blocks, and **embla-carousel** for carousels.

Backend Application Server

- ▶ **Next.js Route Handlers** as the API surface. One HTTP route per domain action, organised by feature folder.
- ▶ **Prisma ORM** over **SQLite** in the current deployment, with a schema annotated for PostgreSQL migration when scale demands it.
- ▶ **JWT sessions** signed with a persistent random secret, **bcrypt** for password hashing, **NextAuth.js v4** available for federated flows.
- ▶ **Nodemailer** for SMTP delivery, **Resend** as a managed email provider, **Ethereal** as a test inbox with preview URLs.
- ▶ **Stripe** and **Razorpay** for payments, with webhook reconciliation and a customer portal.
- ▶ **Cheerio** for server side HTML parsing in the enrichment and scraping paths that run inside the main server.
- ▶ **pdfkit** for PDF generation, **sharp** for image processing, **uuid** for identifiers, **date-fns** for date math.
- ▶ **jose** and **jsonwebtoken** for token signing and verification across the auth, relay, and OAuth flows.
- ▶ **iredis** for caching and pub/sub where a Redis instance is available, with an in memory fallback for single host deployments.

Python Service Layer

The Python layer exists to do what Python does better than TypeScript: heavy scraping at scale, ML model serving, and data science pipelines. These services run as independent processes behind the same gateway, addressed by port, and called from the main application server over HTTP. The architecture is polyglot by design, the right language for the right job, glued together by a single routing convention.

- ▶ **FastAPI** as the web framework for each Python microservice, chosen for async support, automatic OpenAPI docs, and Pydantic validation.
- ▶ **Scrapy** for distributed web crawling, with retry middleware, proxy rotation, and per spider rate limiting.
- ▶ **BeautifulSoup4** and **lxml** for HTML parsing where Scrapy is too heavy for a single page extraction.
- ▶ **Requests** and **httpx** for HTTP clients, with connection pooling and retry sessions.
- ▶ **Pandas** and **NumPy** for data shaping, deduplication, and the statistical work behind the scoring and benchmarking models.
- ▶ **scikit-learn** for the classical ML models that power reply classification and lead scoring baselines.
- ▶ **Celery** with **Redis** as the broker for asynchronous scraping and enrichment jobs that run longer than a single HTTP request.
- ▶ **OpenAI** and **Anthropic** Python SDKs for LLM calls that benefit from the Python ecosystem, such as batch scoring and structured output.
- ▶ **LangChain** for chain and agent orchestration in the more complex AI workflows, with LangSmith hooks for tracing when enabled.
- ▶ **Pydantic** for every service boundary, so the contract between the Python layer and the TypeScript layer is explicit and validated.
- ▶ **Uvicorn** and **Gunicorn** as the ASGI servers, with workers tuned to the available CPU.

AI And ML

- ▶ **z-ai-web-dev-sdk** for LLM chat, vision (VLM), text to speech (TTS), speech to text (ASR), and image generation, used server side only.
- ▶ An in process **vector search** index for retrieval augmented generation over ingested documents and URLs.
- ▶ A **RAG ingestion** pipeline that accepts text, URLs, and CSV, chunks and embeds the content, and stores it for later retrieval.
- ▶ A **prompt manager** that versions and tests prompts, so changes to AI behaviour are auditable and reversible.
- ▶ A **credit enforcement** layer that gates every AI call against the user credit balance and logs the cost.
- ▶ An **AI cost tracker** that aggregates spend per user, per model, and per action for transparency and billing.

Real Time And Messaging

- ▶ **Socket.io** for WebSocket based real time updates, run as a dedicated mini service on its own port.
- ▶ **Server Sent Events** for one way streaming, such as payment status and AI chat token streaming.
- ▶ An in process **event bus** for decoupled communication between services on the same host.
- ▶ A **notification engine** that fans a single event out to email, push, Telegram, and WhatsApp based on user preferences.

Integrations

Integration	Purpose	Library / API
Gmail	Inbox sync, send, reply, tracking, multi account	Gmail API, OAuth2, PubSub push
Google Calendar	Availability, booking, reminders, watches	Google Calendar API, OAuth2, webhook push
Telegram	Notifications and bot interactions	Telegram Bot API
WhatsApp	Messaging via Meta Cloud API and Twilio	Meta Cloud API, Twilio Messaging API
Stripe	Subscriptions, one time payments, customer portal	Stripe SDK, webhook reconciliation
Razorpay	India domestic payments and UPI	Razorpay SDK, webhook reconciliation
Resend	Transactional email delivery	Resend SDK
Google Custom Search	Discovery queries	Google Custom Search JSON API
Sentry	Error tracking and performance monitoring	Sentry SDK

Infrastructure

- ▶ **Caddy** as the gateway, with automatic TLS and a query parameter based port routing convention for mini services.
- ▶ **SQLite** as the default database, file based, zero administration, and sufficient for the working set of a typical revenue team.
- ▶ **Prisma** with a schema annotated for **PostgreSQL** migration, so the move to a managed Postgres is a configuration change, not a rewrite.
- ▶ **Redis** optional, for caching and Celery brokering, with graceful in memory fallback when absent.
- ▶ **Bun** as the JavaScript runtime for the mini services, for fast startup and low overhead.
- ▶ **Node.js** for the main Next.js server, the current production runtime.
- ▶ **ESLint** and **Vitest** for code quality and tests, with a security scan script wired into the build.

SECTION 08

Delivery Discipline

A platform is only as credible as the discipline behind it. AcquisitionOS is engineered with a set of explicit delivery practices that are visible in the codebase, the commit history, and the running system. These are not aspirational policies, they are the operating rules the system is built under every day.

The Practices We Follow

Single Source Of Truth. One database, one schema, one ORM. Every record knows about every other record. There is no shadow data store, no cached copy that drifts, no second CRM that the finance team maintains in parallel. When a record moves, every consumer of that record sees the move in the same transaction.

Every Action Is Audited. Every write to the database is accompanied by an audit log entry that records who, what, when, and from where. The audit log is its own table, append only, and queryable from the admin console. A suspicious login, a stage change, a credit adjustment, an API key rotation, all are traceable to a source.

No Silent Failures. Every error is logged with context. Every catch block either handles the error meaningfully or rethrows it with a structured message. The system never swallows an error and continues as if nothing happened. The observability layer surfaces failure rates per route, so a degrading endpoint is visible before it becomes an outage.

Compliance By Construction. GDPR export and delete, consent tracking, data retention, and unsubscribe handling are built into the data model, not added later. A lead record carries its consent state. An unsubscribe request removes the lead from active sequences immediately. A GDPR delete purges the record and its dependents in a single transaction.

Credits Are Real. Every AI call, every scrape, every outbound message consumes a credit, and every consumption is logged against a ledger. The user can see, in real time, what each action cost. There is no hidden spend, no surprise bill at the end of the month. The credit ledger is its own auditable sub system.

Secrets Stay Secret. Secrets are never committed. The environment is restored on every cold start from a guarded script. Placeholder detection prevents fake credentials from being mistaken for real ones. The system refuses to send email or start OAuth with placeholder credentials rather than failing silently in production.

Backwards Compatible Schema. The Prisma schema is annotated with migration safety notes. Every field that would need a backfill is marked. Every foreign key that would need integrity verification is

marked. The move from SQLite to PostgreSQL is a planned operation, not an emergency one.

Tested Where It Matters. The build includes a security scan. The auth, billing, and pipeline paths carry explicit test coverage. Vitest is wired in and run as part of the development loop. The system is not tested to exhaustion, but it is tested where the cost of a bug is highest.

Observable From Day One. Structured logging, metrics, tracing, health checks, and an error tracker are wired into every route. The observability dashboard is a first class surface in the admin console. The team does not need a third party to tell it what the system is doing.

Documented In Place. Every module has a header comment that explains what it does and why. Every non obvious decision has a comment that records the reasoning. The worklog is a living document that records what was changed, why, and what was verified. A new engineer can read the codebase and understand it without a separate wiki.

The Standard We Hold Ourselves To

If a reviewer, a client, or an auditor asks why a decision was made, the answer must exist in the code, the commit history, or the worklog. If the answer is not there, it is a bug in our discipline, and we fix the discipline before we fix the code.

SECTION 09

Field Notes / Verified

These are the facts about the system as it stands today, verified against the running code and the recorded worklog. They are not promises, they are observations. Where something is not yet live, it says so plainly.

What Is Verified Working

- ❓ The application server builds, starts, and serves HTTP 200 on the root route and on the health check endpoint.
- ❓ The authentication system supports four real sign in paths: email and password, magic link, one time password, and Google OAuth. All four deliver through real protocol, not simulation. No dev tokens, no fake OTPs, no auto complete.
- ❓ Email delivery runs through a real SMTP chain, Nodemailer to Resend or Gmail SMTP, with Ethereal as a test inbox that produces a browser viewable preview URL when no production credentials are set.
- ❓ The database schema covers over sixty domain models, including users, organisations, leads, deals, pipeline stages, outreach sequences, meetings, workflows, credits, invoices, and GDPR records. The schema is annotated for PostgreSQL migration.
- ❓ The API surface covers every module listed in this document, with one route per action, rate limiting, and audit logging. The route count is in the hundreds, organised by feature folder.
- ❓ The real time layer runs as a dedicated Socket.io mini service and pushes live updates to connected dashboard clients.
- ❓ The billing layer integrates Stripe and Razorpay with webhook reconciliation, invoice generation, tax calculation, and a customer portal. Coupons, trials, and credit addons are supported.
- ❓ The integration hub supports Gmail, Google Calendar, Telegram, WhatsApp (Meta and Twilio), Stripe, and Razorpay, each with connect, disconnect, and status endpoints.
- ❓ The observability layer produces structured logs, metrics, and health checks. Sentry is wired in with a no op fallback when no DSN is configured.
- ❓ The workflow engine supports triggers, conditions, actions, executions, retries, and a dead letter queue, with per workflow metrics.
- ❓ The AI copilot supports streaming chat, RAG over ingested documents, vector search, and credit enforced calls. The AI cost tracker aggregates spend per action.
- ❓ The security layer includes rate limiting, input validation, CSRF protection, security headers, webhook signature verification, device fingerprinting, suspicious login detection, and an audit log.
- ❓ The admin console includes feedback management, billing analytics, crash reports, backup management, and refund handling.
- ❓ The system is deployable as a standalone Next.js build, with a Dockerfile that expects the standalone output. The build has been verified to produce the standalone server entry point.

What Is In Progress

- Production email delivery to end user inboxes is pending a live Gmail App Password being set on the server. The code path is complete, the resolver supports fourteen alias names for the SMTP password, and delivery activates the moment a real password is set under any alias.
- The Python service layer is specified and the gateway routing convention is in place, but the individual Python microservices are on the integration roadmap. The TypeScript scraping and AI paths are live in the interim.
- The PostgreSQL migration path is annotated and the schema is ready, but the current deployment runs on SQLite. The move to PostgreSQL is a configuration change, not a rewrite, and is scheduled to follow the first enterprise deployment.
- The competitor intelligence module is implemented and the data model is live, but the scheduled snapshot collection is pending a live scraping budget allocation.
- The white label branding service is implemented at the data model and API level, and the dashboard respects per organisation branding, but the self serve branding studio is on the UI roadmap.

What Is Explicitly Out Of Scope Today

- A mobile native application. The dashboard is a responsive web application and works on mobile browsers, but no native iOS or Android client is shipped.
- A self serve signup flow for the general public. Sign up is available but the onboarding is designed for a guided first use, not a frictionless consumer funnel.
- An offline mode. The system assumes connectivity. Offline recovery exists for the client, but the server is online only.

Development Status

AcquisitionOS is in active development. The platform is functional, the core modules are live, and the system is being prepared for its first external pilot deployments. The items marked in progress are on the near term roadmap. The items marked out of scope are deliberate non goals for the current phase. This document describes the system as it stands today, not as a promise of what it might become.

SECTION 10

In Summary

AcquisitionOS is a single platform that replaces a scattered revenue stack with one coherent operating system. It discovers prospects, scores them with AI, reaches out across channels, books meetings, moves deals through a pipeline, reports revenue, and bills the customer, all in one database, one login, one audit trail. The system is built with mainstream technology, operated by a small team, and designed to be defensible to an enterprise buyer. It is in active development, the core is live, and it is being prepared for its first pilot deployments. For a revenue team that is tired of operating its tools instead of its pipeline, this is the alternative.

At A Glance

1 PLATFORM One login, one database, one bill	60+ DOMAIN MODELS In a single Prisma schema	4 AUTH PATHS Password, magic link, OTP, Google	6+ INTEGRATIONS Gmail, Calendar, WhatsApp, Telegram, Stripe, Razorpay
---	--	---	--

Document Control

Field	Value
Document	AcquisitionOS Corporate Capability Document
Edition	v2.0 / 2026
Status	In Development
Audience	Corporate website listing and client deal review
Classification	Confidential, authorised distribution only
Source	AcquisitionOS engineering worklog and running codebase

End of document.